

设计原则速记

总口诀

内部暴露 -> 信息隐藏、迪米特法则

职责混乱 -> SRP、高内聚差

直接依赖实现 -> DIP

新增规则总改老代码 -> OCP

接口太大 -> ISP

替换实现出错 -> LSP

包互相依赖 -> ADP

重复逻辑 -> DRY

设计太复杂 -> KISS

提前做没用功能 -> YAGNI

信息隐藏

核心： 模块只暴露必要接口，隐藏内部数据结构和实现细节。

题干信号：

- 直接访问内部对象
- 暴露内部数据结构
- 字段变化影响多个包

答题句：

违反信息隐藏原则，因为内部数据结构和变化点没有被封装，其他模块可以直接依赖内部实现。

SRP：单一职责原则

核心： 一个类或包只负责一类职责，只有一个主要变化理由。

题干信号：

- 职责边界不清
- 一个类或包做太多事
- 业务规则、通知、资源状态、用户权限混在一起

答题句：

违反 SRP，因为包和类承担多个职责，存在多个变化理由。

OCP：开闭原则

核心： 对扩展开放，对修改关闭。

题干信号：

- 新增规则要修改原有主流程
- 变化点没有封装
- 每加一种功能都要改旧代码

答题句：

违反 OCP，因为新增业务规则时需要修改已有主流程，而不是通过新增扩展类完成。

例子：

预约冲突检测规则可以抽成 `ConflictCheckStrategy`。新增时间段冲突、容量冲突、权限冲突、设备状态冲突时，新增策略实现类。

LSP：里氏替换原则

核心： 子类对象应该可以替换父类对象，并且程序行为不被破坏。

题干信号：

- 子类替换父类后行为不一致
- 实现类不遵守接口语义
- 换一个实现后业务出错

答题句：

违反 LSP，因为子类或实现类不能在不破坏原有行为的情况下替换父类或接口。

例子：

ReservationRepository 的内存实现和数据库实现都应该遵守 save 的语义。

按资源和时间段查询已有预约的方法，也应该在不同实现中保持相同语义。

ISP：接口隔离原则

核心： 接口要小而专一，不要让调用方依赖自己不用的方法。

题干信号：

- 接口太大
- 一个接口塞很多职责
- 调用方被迫依赖不需要的方法

答题句：

违反 ISP，因为接口职责过大，调用方依赖了自己不需要的的方法。

例子：

不要把预约、工单、通知、用户、资源都塞进 CampusService。应该拆成 ReservationService、WorkOrderService、NotificationGateway、UserService、ResourceQueryService。

DIP：依赖倒置原则

核心： 高层业务不直接依赖低层实现，双方都依赖抽象接口。

题干信号：

- 高层业务直接依赖具体实现
- 直接 new 低层对象
- 直接访问低层内部对象

答题句：

违反 DIP，因为高层业务直接依赖低层实现对象，而不是依赖稳定接口。

正确方向：

ReservationService -> ReservationRepository 接口

MysqlReservationRepository -> 实现 ReservationRepository

错误方向:

ReservationService -> MysqlReservationRepository

Law of Demeter: 迪米特法则

核心: 一个对象只和直接相关的对象通信, 不要穿透别人内部结构。

题干信号:

- 跨对象访问内部结构
- 链式访问过多
- 知道太多别的对象细节

答题句:

违反 Law of Demeter, 因为对象跨越边界访问其他对象的内部结构, 知道了过多实现细节。

坏味道:

`a.getB().getC().getD()`

ADP: 无环依赖原则

核心: 包之间的依赖关系不能形成环。

题干信号:

- 包互相依赖
- 循环依赖
- A 依赖 B, B 又依赖 A

答题句:

违反 ADP, 因为包依赖关系中存在环, 修改一个包会牵连其他包。

错误例子:

`reservation -> resource`

`resource -> reservation`

DRY：不要重复

核心： 同一知识或规则只保留一份。

题干信号：

- 同一逻辑多处复制
- 规则散落在多个地方
- 改一处还要同步改多处

答题句：

违反 DRY，因为相同规则在多个位置重复出现，导致修改成本和不一致风险增加。

KISS：保持简单

核心： 能简单解决，就不要设计得过度复杂。

题干信号：

- 设计过度
- 简单问题复杂化
- 结构很绕但收益不明显

答题句：

违反 KISS，因为设计复杂度超过当前问题需要，增加了理解和维护成本。

YAGNI：不要提前设计用不到的东西

核心： 当前没有明确需求的功能，不要提前实现。

题干信号：

- 为了未来功能提前写大量代码
- 当前需求用不到
- 过早抽象

答题句：

违反 YAGNI，因为实现了当前需求并不需要的功能或抽象。

高内聚、低耦合

核心： 高内聚是一个模块内部的内容围绕同一职责组织。低耦合是模块之间尽量少知道彼此细节。

题干信号：

- 包之间互相直接访问内部对象 -> 低耦合被破坏
- 多个职责混在不同包中 -> 高内聚被破坏
- 共享可变全局配置 -> 公共耦合

第六题可用答案：

从耦合角度看，各包互相直接访问内部对象，导致高访问耦合和循环依赖；多个包共享同一个可变全局配置对象，形成公共耦合。

从内聚角度看，预约、资源、工单、通知和用户权限等职责混在不同包中，职责边界不清，导致模块内聚性下降。